

Assignment 3

Aim : Write a program to implement the solution of fractional knapsack

```
#include <stdio.h>
```

```
struct Entry {
    int v, w;
    float r;
} a[100], t[100];
```

```
void merge(int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high) {
        t[k++] = (a[i].r > a[j].r) ? a[i++] : a[j++];
    }
    while (i <= mid) {
        t[k++] = a[i++];
    }
    while (j <= high) {
        t[k++] = a[j++];
    }
    for (i = low; i <= high; i++) {
        a[i] = t[i];
    }
}
```

```
void mergeSort(int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(low, mid);
        mergeSort(mid + 1, high);
        merge(low, mid, high);
    }
}
```

```

void main() {
    int n, c;
    float ans;
    printf("Enter number of items: ");
    scanf("%d", &n);
    for(int i=0; i<n; i++) {
        printf("Enter value & weight: ");
        scanf("%d %d", &a[i].v, &a[i].w);
        a[i].r = (float) a[i].v / a[i].w;
    }

    printf("Enter capacity: ");
    scanf("%d", &c);
    mergeSort(0, n-1);
    for(int i=0; i<n && c>0; i++) {
        if(a[i].w <= c) {
            ans += a[i].v;
            c -= a[i].w;
        } else {
            ans += a[i].r * c;
            c = 0;
        }
    }

    printf("Max value: %.2f", ans);
}

```

Function	Time complexity	Space complexity
merge()	$O(n)$	$O(n)$
mergeSort()	$O(n \log n)$	$O(\log n)$

```
Enter number of items: 3
Enter value & weight: 60 10
Enter value & weight: 100 20
Enter value & weight: 120 30
Enter capacity: 50
Max value: 240.00
Process returned 17 (0x11)   execution time : 22.471 s
Press any key to continue.
|
```